

AI Assisted Coding

Lab Assignment 11.3

Name: CH,MANMITH VARDHAN

Hall Ticket no: 2303A51321

Batch No: 19

Task -1:

Prompt:

generate a Python Contact Manager using both array (list) and linked list data structures. The system should support adding, searching, deleting, and displaying contacts. I also requested case-insensitive search functionality and clear output messages. The goal was to compare static and dynamic data storage approaches.

```
3
4 # Task- 1: Smart Contact Manager
5 # Array-based Contact Manager
6 class ArrayContactManager:
7     def __init__(self):
8         self.contacts = []
9
10    def add_contact(self, name, phone):
11        self.contacts.append({"name": name, "phone": phone})
12        print(f"Contact '{name}' added.")
13
14    def search_contact(self, name):
15        for contact in self.contacts:
16            if contact["name"].lower() == name.lower():
17                return contact
18        return None
19
20    def delete_contact(self, name):
21        for i, contact in enumerate(self.contacts):
22            if contact["name"].lower() == name.lower():
23                self.contacts.pop(i)
24                print(f"Contact '{name}' deleted.")
25                return True
26        return False
27
28    def display_contacts(self):
29        if not self.contacts:
30            print("No contacts found.")
31        else:
32            for contact in self.contacts:
33                print(f"{contact['name']}: {contact['phone']}")
34
35
36 # Linked List Node
37 class ContactNode:
38     def __init__(self, name, phone):
39         self.data = {"name": name, "phone": phone}
40         self.next = None
41
42 # Linked List-based Contact Manager
43 class LinkedListContactManager:
44     def __init__(self):
45         self.head = None
```

OUTPUT:

```
PROBLEMS 2 OUTPUT DEBUG CONSOLE TERMINAL PORTS
PS C:\Users\spand\OneDrive\Desktop\ai assisted coding> & C:/Users/spand/AppData/Local/Programs/Python/Python11.3-Assignment.py"
=== PERFORMANCE COMPARISON ===

Array-based Contact Manager:
- Insertion: O(1) - append at end
- Deletion: O(n) - must shift elements
- Search: O(n)

Linked List-based Contact Manager:
- Insertion: O(n) - traverse to end
- Deletion: O(n) - traverse to find node
- Search: O(n)

=== DEMO ===

Contact 'spandana' added.
Contact 'harshit' added.
Search: {'name': 'spandana', 'phone': '8374212306'}
Contact 'spandana' deleted.

Contact 'raju' added.
Contact 'vinodha' added.
Search: {'name': 'raju', 'phone': '9963212306'}
Contact 'raju' deleted.
PS C:\Users\spand\OneDrive\Desktop\ai assisted coding> |
```

Justification:

Arrays are simple and allow fast insertion using append operations, making them suitable for small datasets. Linked lists provide dynamic memory allocation and efficient deletion without shifting elements. Both structures help understand different memory and performance behaviors. This comparison improves understanding of data structure efficiency.

Task 2:

Prompt: create a library request system using a FIFO queue and a priority queue in Python. The implementation needed enqueue and dequeue operations using deque and heapq. Faculty requests were required to have higher priority than student requests. The system should demonstrate request processing order.

```

113 # Task 2: Library Book Search System (Queues & Priority Queues)
114
115 # Simple Queue for FIFO requests
116 class BookRequestQueue:
117     def __init__(self):
118         self.queue = deque()
119
120     def enqueue(self, name, book_title, request_type):
121         self.queue.append({"name": name, "book_title": book_title, "type": request_type})
122         print(f"Request added: {name} ({request_type}) - {book_title}")
123
124     def dequeue(self):
125         if self.queue:
126             request = self.queue.popleft()
127             print(f"Processing: {request['name']} - {request['book_title']}")
128             return request
129         print("No requests in queue.")
130         return None
131
132     def display_queue(self):
133         if not self.queue:
134             print("Queue is empty.")
135         else:
136             for req in self.queue:
137                 print(f"    {req['name']} ({req['type']}) - {req['book_title']}")
138
139 # Priority Queue for Faculty-first requests
140 class BookRequestPriorityQueue:
141     def __init__(self):
142         self.heap = []
143         self.counter = 0
144
145     def enqueue(self, name, book_title, request_type):
146         priority = 0 if request_type == "Faculty" else 1
147         heapq.heappush(self.heap, (priority, self.counter, {"name": name, "book_title": book_title, "type": request_type}))
148         self.counter += 1
149         print(f"Request added: {name} ({request_type}) - {book_title}")
150
151     def dequeue(self):
152         if self.heap:
153             priority, _, request = heapq.heappop(self.heap)
154             print(f"Processing: {request['name']} ({request['type']}) - {request['book_title']}")
155             return request
156         print("No requests in queue.")
157         return None

```

Output

```

manmithvardhanchalla@Manmiths-MacBook-Air AI ASSIS-CODING % /usr/bin/python3 "/Users/manmithvardhanchalla/Desktop/AI ASSIS-CODING/assignment-11.3.py"

=== SIMPLE QUEUE (FIFO) ===
Request added: John (Student) - Python 101
Request added: Dr. Smith (Faculty) - Advanced Algorithms
Request added: Sarah (Student) - Data Science

Queue contents:
John (Student) - Python 101
Dr. Smith (Faculty) - Advanced Algorithms
Sarah (Student) - Data Science

Processing requests:
Processing: John - Python 101
Processing: Dr. Smith - Advanced Algorithms

=== PRIORITY QUEUE (Faculty First) ===
Request added: John (Student) - Python 101
Request added: Dr. Smith (Faculty) - Advanced Algorithms
Request added: Sarah (Student) - Data Science
Request added: Prof. Johnson (Faculty) - Machine Learning

Queue contents:
Dr. Smith (Faculty) - Advanced Algorithms
Prof. Johnson (Faculty) - Machine Learning
John (Student) - Python 101
Sarah (Student) - Data Science

Processing requests (Faculty prioritized):
Processing: Dr. Smith (Faculty) - Advanced Algorithms
Processing: Prof. Johnson (Faculty) - Machine Learning
Processing: John (Student) - Python 101

```

Justification:

A queue ensures fair processing of requests in arrival order using FIFO logic. A priority queue allows urgent requests, such as faculty requests, to be handled first. This combination models real-world service systems effectively. Both structures provide efficient $O(1)$ or $O(\log n)$ operations.

Task-3:

Prompt:

To implement a stack-based ticket management system using Python. The stack needed push, pop, peek, and status-check operations. The implementation should simulate resolving tickets using LIFO behavior. The system also required a demonstration with multiple support tickets.

```
191 # Task - 3: Emergency Help Desk (Stack Implementation)
192 # Stack Node for tickets
193 class TicketNode:
194     def __init__(self, ticket_id, student_name, issue):
195         self.ticket_id = ticket_id
196         self.student_name = student_name
197         self.issue = issue
198         self.next = None
199
200 # Stack-based Ticket Management System
201 class SupportTicketStack:
202     def __init__(self, max_size=100):
203         self.top = None
204         self.size = 0
205         self.max_size = max_size
206
207     def push(self, ticket_id, student_name, issue):
208         if self.is_full():
209             print(f"Stack is full! Cannot add ticket {ticket_id}.")
210             return False
211         new_ticket = TicketNode(ticket_id, student_name, issue)
212         new_ticket.next = self.top
213         self.top = new_ticket
214         self.size += 1
215         print(f"Ticket #{ticket_id} added: {student_name} - {issue}")
216         return True
217
218     def pop(self):
219         if self.is_empty():
220             print("No tickets to resolve.")
221             return None
222         ticket = self.top
223         self.top = self.top.next
224         self.size -= 1
225         print(f"Resolving Ticket #{ticket.ticket_id}: {ticket.student_name} - {ticket.issue}")
226         return ticket
227
228     def peek(self):
229         if self.is_empty():
230             print("No tickets in queue.")
231             return None
232         return self.top
233
234     def is_empty(self):
235         return self.size == 0
```

Output:

```
manmithvardhanchalla@Manmiths-MacBook-Air AI ASSIS-CODING % /usr/bin/python3 "/Users/manmithvardhanchalla/Desktop/AI ASSIS-CODING/assignment-11.3.py"
=== EMERGENCY HELP DESK (STACK) ===

Adding support tickets:

Ticket #101 added: Alice Johnson - Laptop won't connect to WiFi
Ticket #102 added: Bob Smith - Password reset failed
Ticket #103 added: Carol White - Software installation issue
Ticket #104 added: David Brown - Email not syncing
Ticket #105 added: Emma Davis - Printer driver problem

Current pending tickets:
Pending Tickets (LIFO order):
  1. Ticket #105: Emma Davis - Printer driver problem
  2. Ticket #104: David Brown - Email not syncing
  3. Ticket #103: Carol White - Software installation issue
  4. Ticket #102: Bob Smith - Password reset failed
  5. Ticket #101: Alice Johnson - Laptop won't connect to WiFi

Peek at next ticket to resolve:
  Next: Ticket #105

Resolving tickets (LIFO - Last In, First Out):

Resolving Ticket #105: Emma Davis - Printer driver problem
Resolving Ticket #104: David Brown - Email not syncing
Resolving Ticket #103: Carol White - Software installation issue
e

Remaining tickets:
Pending Tickets (LIFO order):
  1. Ticket #102: Bob Smith - Password reset failed
  2. Ticket #101: Alice Johnson - Laptop won't connect to WiFi

Stack status - Empty: False, Full: False
```

Justification:

A stack follows Last-In First-Out (LIFO), which matches emergency escalation handling. Recently raised issues are resolved first during urgent situations. Stack operations are efficient and simple to implement. This structure clearly demonstrates real-world problem prioritization.

Task-4:

Prompt:

Generate a hash table implementation in Python with insert, search, and delete operations. The solution needed collision handling using chaining. The code should include comments explaining hashing and indexing. The goal was to understand fast data retrieval techniques.

```
281 # Task - 4: # Hash Table with Chaining for Collision Handling
282 class HashTable:
283     def __init__(self, size=10):
284         #Initialize hash table with given size
285         self.size = size
286         self.table = [[] for _ in range(size)]
287
288     def _hash(self, key):
289         #Generate hash value for key using simple modulo function
290         return hash(key) % self.size
291
292     def insert(self, key, value):
293         #Insert key-value pair into hash table
294         hash_index = self._hash(key)
295
296         # Check if key already exists and update it
297         for i, (k, v) in enumerate(self.table[hash_index]):
298             if k == key:
299                 self.table[hash_index][i] = (key, value)
300                 print(f"Updated: {key} -> {value}")
301                 return
302
303         # Add new key-value pair (chaining handles collision)
304         self.table[hash_index].append((key, value))
305         print(f"Inserted: {key} -> {value}")
306
307     def search(self, key):
308         #Search for value by key
309         hash_index = self._hash(key)
310
311         # Linear search through chain at hash index
312         for k, v in self.table[hash_index]:
313             if k == key:
314                 print(f"Found: {key} -> {v}")
315                 return v
316
317         print(f"Not found: {key}")
318         return None
319
320     def delete(self, key):
321         #Delete key-value pair from hash table
322         hash_index = self._hash(key)
323
324         # Search and remove from chain
325         for i, (k, v) in enumerate(self.table[hash_index]):
```

Output:

```

manmithvardhanchalla@Manmiths-MacBook-Air AI ASSIS-CODING % /usr/bin/python3 "/Users/manmithvardhanchalla/Desktop/AI ASSIS-CODING/assignment-11.3.py"
Stack status - Empty: False, Full: False
=== HASH TABLE WITH CHAINING ===

Inserting entries:

Inserted: name -> Alice
Inserted: age -> 25
Inserted: city -> NYC
Inserted: job -> Engineer
Updated: age -> 26

Searching for entries:

Found: name -> Alice
Found: age -> 26
Not found: country

Hash table structure:
Hash Table Contents:
  Index 0: [('age', 26), ('job', 'Engineer')]
  Index 3: [('city', 'NYC')]
  Index 4: [('name', 'Alice')]

Deleting entries:

Deleted: city
Key not found: country

Final hash table:
Hash Table Contents:
  Index 0: [('age', 26), ('job', 'Engineer')]
  Index 4: [('name', 'Alice')]

```

Justification:

Hash tables allow very fast data access using key-based indexing. Collision handling through chaining prevents data loss when keys map to the same index. This structure provides average $O(1)$ search performance. It is widely used in databases and real-time lookup systems.

Task-5:

Prompt:

design a campus management system using suitable data structures for different features such as attendance, event registration, library borrowing, bus scheduling, and cafeteria orders. One feature needed full implementation. The solution also required justification for each structure selection.

```

367 # Task - 5:
368 # Real-Time Campus Resource Management System
369
370 # Feature 1: Student Attendance Tracking - Using Dictionary (Hash Map)
371 # Justification: O(1) average lookup and insertion for student records.
372 # Perfect for quick attendance marking and retrieval by student ID.
373
374 class AttendanceSystem:
375     def __init__(self):
376         self.attendance = {}
377
378     def mark_attendance(self, student_id, date, status):
379         if student_id not in self.attendance:
380             self.attendance[student_id] = {}
381         self.attendance[student_id][date] = status
382         print(f"Marked {student_id} as {status} on {date}")
383
384     def get_attendance(self, student_id):
385         if student_id in self.attendance:
386             print(f"Attendance for {student_id}:")
387             for date, status in self.attendance[student_id].items():
388                 print(f"    {date}: {status}")
389             return self.attendance[student_id]
390         print(f"No records for {student_id}")
391         return None
392
393     def get_attendance_percentage(self, student_id):
394         if student_id not in self.attendance:
395             return 0
396         records = self.attendance[student_id]
397         present = sum(1 for s in records.values() if s == "Present")
398         percentage = (present / len(records)) * 100 if records else 0
399         print(f"{student_id} attendance: {percentage:.1f}%")
400         return percentage
401
402 # Feature 2: Event Registration System - Using Set
403 # Justification: Sets provide O(1) lookup and prevent duplicate registrations.
404 # No duplicates allowed; perfect for managing unique student registrations.
405
406 class EventRegistration:
407     def __init__(self, event_name):
408         self.event_name = event_name
409         self.registered_students = set()
410
411     def register(self, student_id):

```

Output:


```

manmithvardhanchalla@Manmiths-MacBook-Air AI ASSIS-CODING % /usr/bin/python3 "/Users/manmithvardhanchalla/Desktop/AI ASSIS-CODING/assignment-11.3.py"

--- CAFETERIA ORDER QUEUE (Selected Feature) ---

Order #1 placed for S001: ['Pizza', 'Coke']
Order #2 placed for S002: ['Burger', 'Fries']
Order #3 placed for S003: ['Salad', 'Water']

Pending orders:
Pending Orders:
#1: S001 - ['Pizza', 'Coke']
#2: S002 - ['Burger', 'Fries']
#3: S003 - ['Salad', 'Water']

Processing orders:
Order #1 completed for S001
Order #2 completed for S002

Remaining orders:
Pending Orders:
#3: S003 - ['Salad', 'Water']

--- OTHER FEATURES DEMO ---

Marked S001 as Present on 2024-01-15
Marked S001 as Absent on 2024-01-16
S001 attendance: 50.0%
S001 registered for Tech Conference
S002 registered for Tech Conference
Tech Conference: 2 students registered
S001 borrowed 'Python Guide' (Due: 2024-02-01)
Books borrowed by S001:
- Python Guide (Due: 2024-02-01)
Route B1 added: 08:00
All Bus Routes:
B1: ['Gate', 'Hall A', 'Library'] - Departs: 08:00
manmithvardhanchalla@Manmiths-MacBook-Air AI ASSIS-CODING %

```

Justification:

Different system components require different data structures for efficiency. Dictionaries provide fast attendance lookup, sets prevent duplicate registrations, and queues manage ordered requests. Selecting appropriate structures improves performance and scalability. This demonstrates practical application of data structures in real systems.